

MAJOR PROJECT REPORT

DEVOPS PIPELINE OPTIMIZATION SYSTEM

Submitted in Partial Fulfillment of the Requirements for the Degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

Third Semester

Academic Session: 2026–2027

Submitted To

School of Computer Application and Technology

Galgotias University

Greater Noida, Uttar Pradesh, India



Under the Guidance of

Dr. Deepak Kumar Panda

Submitted By

Vishal Yadav | Admission No.: 25SCSE2030186 | Enrollment No.: 25132030877

Satyam Tiwari | Admission No.: 25SCSE2030189 | Enrollment No.: 25132030794

Rohit Baghel | Admission No.: 25SCSE2030104 | Enrollment No.: 25132030769

DECLARATION

We hereby declare that the project report titled “DevOps Pipeline Optimization System” has been prepared and submitted by us as a part of the requirements for the Master of Computer Applications (MCA), Third Semester, at School of Computer Application and Technology, Galgotias University.

This project has been completed under the guidance of Dr. Deepak Kumar Panda. The project is based on our own work and learning during the development of the system.

The project demonstrates an automated delivery setup for a Python Flask application using Git, GitHub, Docker, Terraform, GitHub Actions, Azure Container Registry, and Azure App Service. The implementation details in this report are based on the supplied project files and execution screenshots.

We have prepared this report for academic purposes and have presented the available project work as accurately as possible.

S. No.	Team Member	Admission No.	Enrollment No.
1	Vishal Yadav	25SCSE2030186	25132030877
2	Satyam Tiwari	25SCSE2030189	25132030794
3	Rohit Baghel	25SCSE2030104	25132030769

Signatures: _____ Date: _____ Place: Greater Noida,
Uttar Pradesh

ACKNOWLEDGEMENT

We would like to express our sincere thanks to Galgotias University and the School of Computer Application and Technology for giving us the opportunity to work on this major project.

We are thankful to our project guide, Dr. Deepak Kumar Panda, for his guidance and support during the development of this project. His suggestions helped us understand the practical concepts of DevOps, cloud deployment, Docker, Terraform, and CI/CD.

We also thank the faculty members who provided the academic knowledge and support required to complete the project.

Finally, we are grateful to everyone who directly or indirectly supported the development work and preparation of this report.

ABSTRACT

The DevOps Pipeline Optimization System demonstrates a practical workflow for packaging a Python Flask web application as a Docker container and making it available through Microsoft Azure. The project combines source-code management, containerization, Infrastructure as Code, CI/CD automation, and cloud hosting in a single academic implementation.

The Flask application is maintained in Git and GitHub. Terraform configuration defines the Azure Resource Group, Azure Container Registry (ACR), Linux App Service Plan, and Linux Web App. The GitHub Actions workflow is triggered by a push to the main branch; it checks out the repository, prepares Python 3.9, installs Flask, authenticates to ACR using repository secrets, and builds and pushes the Docker image.

The supplied Azure configuration connects the Linux Web App to the ACR image and exposes container port 5000. The submitted screenshots provide evidence of a successful GitHub Actions run, the Azure Web App resource, the Visual Studio Code project, and the final live web application.

A key implementation detail is that the supplied pipeline.yml does not itself execute Terraform or an explicit Azure App Service deployment command. Terraform is used to provision and configure the Azure resources, while GitHub Actions builds and pushes the container image. The final report therefore describes these responsibilities separately rather than claiming steps that are not present in the workflow file.

Keywords: DevOps, CI/CD, Flask, Python, Git, GitHub, GitHub Actions, Docker, Terraform, Microsoft Azure, Azure Container Registry, Azure App Service.

TABLE OF CONTENTS

Note: The table of contents is generated from the report heading structure and can be updated in Microsoft Word using Update Field.

1. INTRODUCTION

1.1 Introduction to DevOps

DevOps is a software development and delivery approach that connects development activities with operational activities through collaboration and automation. In this project, DevOps practices are applied to source control, containerization, infrastructure management, CI/CD automation, and cloud hosting.

1.2 Continuous Integration and Continuous Delivery/Deployment

Continuous Integration (CI) involves integrating code changes into a shared repository and running automated checks or build steps. Continuous Delivery/Deployment (CD) extends the process toward preparing or releasing application changes. In the supplied project, GitHub Actions provides the automation layer and is configured to start when changes are pushed to the main branch.

1.3 Infrastructure as Code

Infrastructure as Code (IaC) represents cloud infrastructure in configuration files rather than relying only on manual portal configuration. Terraform is used in this project to define the Azure Resource Group, Azure Container Registry, Linux Service Plan, and Linux Web App.

1.4 Problem Statement

Manual application deployment can require repeated steps such as building an image, configuring cloud resources, authenticating to a registry, and publishing the application. The project addresses this by combining Terraform-based infrastructure configuration with a GitHub Actions workflow for container image creation and registry publishing.

1.5 Objectives of the Project

- Develop a simple web application using Python Flask.
- Maintain the project using Git and GitHub.
- Package the Flask application using Docker.
- Define Azure infrastructure using Terraform.
- Automate container image build and push using GitHub Actions.
- Store the Docker image in Azure Container Registry.
- Run the application using Azure App Service.
- Reduce repeated manual steps in the delivery workflow.
- Gain practical understanding of DevOps and cloud deployment tools.

1.6 Scope of the Project

The project focuses on a small Flask application and its cloud delivery path. The supplied implementation covers application code, Git/GitHub source management, Docker packaging, Terraform configuration for Azure resources, GitHub Actions automation for image publishing, ACR storage, and Azure App Service hosting. Advanced testing, monitoring, orchestration, and multi-environment delivery are identified as future extensions rather than current implementation features.

2. TOOLS AND TECHNOLOGIES

Technology	Role in the Project
Python	Used for the Flask application and the Python 3.9 runtime in the GitHub Actions workflow.
Flask	Lightweight Python web framework used to create the application and its root route.
Git	Version-control system used to track project changes.
GitHub	Remote repository platform used to store the project and host the GitHub Actions workflow.
GitHub Actions	Automation service used by the supplied pipeline to checkout code, set up Python, install Flask, authenticate to ACR, and build/push the Docker image.
Docker	Containerization platform used to package the Flask application into an image.
Terraform	Infrastructure as Code tool used to define the Azure resources and web-app container configuration.
Microsoft Azure	Cloud platform hosting the Azure Container Registry and Linux Web App resources used by the project.
Azure Container Registry	Private container registry used to store the Docker image <code>mca-devopsreg2026.azurecr.io/mca-devops-app:latest</code> .
Azure App Service	Cloud hosting service configured as a Linux Web App that uses the ACR image and exposes port 5000.
Visual Studio Code	Development environment visible in the supplied project-structure screenshot and used to manage the project files.

3. SYSTEM WORKFLOW

3.1 Actual Workflow Derived from the Supplied Files

The implementation can be understood as two connected paths: (1) infrastructure configuration through Terraform and (2) application image delivery through GitHub Actions. The supplied Terraform file creates/configures the Azure resources, including the Linux Web App and its ACR image settings. The supplied GitHub Actions workflow then builds and pushes the image to ACR when code is pushed to main.

Developer / VS Code	Git	GitHub repository	GitHub Actions	Docker build	Azure Container Registry	Azure Linux Web App	Live Flask application
---------------------	-----	-------------------	----------------	--------------	--------------------------	---------------------	------------------------

Infrastructure path: Terraform configuration → Azure Resource Group + ACR + Linux Service Plan + Linux Web App configuration. Application delivery path: GitHub push → GitHub Actions → Docker build → ACR push → Web App consumes the configured image.

3.2 Step-by-Step Workflow

Application development: The Flask application is written in `app.py` and listens on `0.0.0.0:5000`.

Version control: Project files are maintained in Git and the GitHub repository.

Workflow trigger: `pipeline.yml` is configured for push events on the main branch.

Runner setup: GitHub Actions uses an Ubuntu runner and sets up Python 3.9.

Dependency installation: The workflow upgrades pip and installs Flask.

Registry authentication: The workflow logs in to mcadevopsreg2026.azurecr.io using ACR_USERNAME and ACR_PASSWORD secrets.

Docker image build: The workflow builds the image tagged as mcadevopsreg2026.azurecr.io/mca-devops-app:latest.

Image push: The image is pushed to Azure Container Registry.

Azure hosting: Terraform configures the Linux Web App to use the ACR image and sets WEBSITES_PORT to 5000.

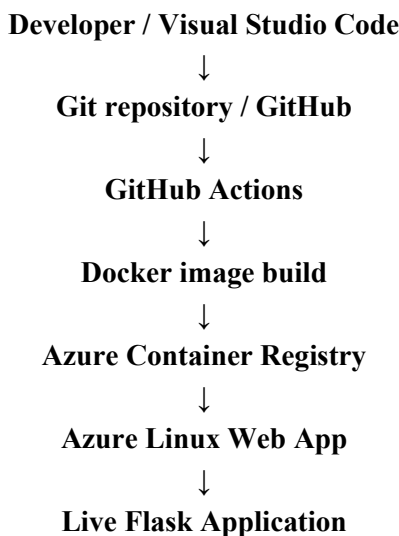
Live output: The supplied browser screenshot shows the deployed application at the Azure App Service domain.

4. SYSTEM ARCHITECTURE

4.1 Architecture Description

The architecture consists of the development environment, source repository, CI/CD automation, container build process, container registry, and Azure web-hosting layer. Terraform defines the cloud infrastructure and configures the Linux Web App to use the container image in ACR. GitHub Actions is responsible for building and pushing the image.

4.2 Architecture Flow



4.3 Main Components

Developer Environment: Visual Studio Code is used to view and manage the project files.

GitHub Repository: Stores the source code and project configuration.

GitHub Actions: Runs the configured automation workflow on pushes to main.

Terraform: Defines Azure resources and the container settings for the Linux Web App.

Docker: Packages app.py into a Python 3.9-slim based container image.

Azure Container Registry: Stores the pushed image.

Azure Linux Web App: Runs the configured container and exposes the Flask application.

5. PROJECT IMPLEMENTATION

5.1 Flask Application

The main application is contained in app.py. It imports Flask, creates the application object, defines the root route, returns the project status page, and starts the development server on host 0.0.0.0 and port 5000. The supplied browser screenshot corresponds to the HTML response produced by this route.

5.2 Docker Implementation

The Dockerfile uses python:3.9-slim as the base image, sets /app as the working directory, installs Flask, copies app.py into the image, exposes port 5000, and starts the application with python app.py.

5.3 Terraform Implementation

main.tf defines the Azure provider and four major resources: Resource Group, Azure Container Registry, Linux Service Plan, and Linux Web App. The Web App is configured with the ACR login server and image name and receives the registry credentials and WEBSITES_PORT setting through app settings. variable.tf supplies the Central India region and the project resource names.

5.4 GitHub Actions Implementation

The supplied pipeline.yml is triggered by push events on main. It runs on ubuntu-latest, checks out the code, sets up Python 3.9, installs Flask, logs into ACR, builds the Docker image, and pushes the image. The file does not contain Terraform commands or an explicit Azure App Service deployment action; those infrastructure and hosting settings are defined in Terraform.

5.5 Visual Studio Code Project Structure

The supplied VS Code screenshot shows the project workspace and the Terraform directory. The visible project includes the GitHub workflow, Terraform configuration, application code, Dockerfile, README, and variable definitions.

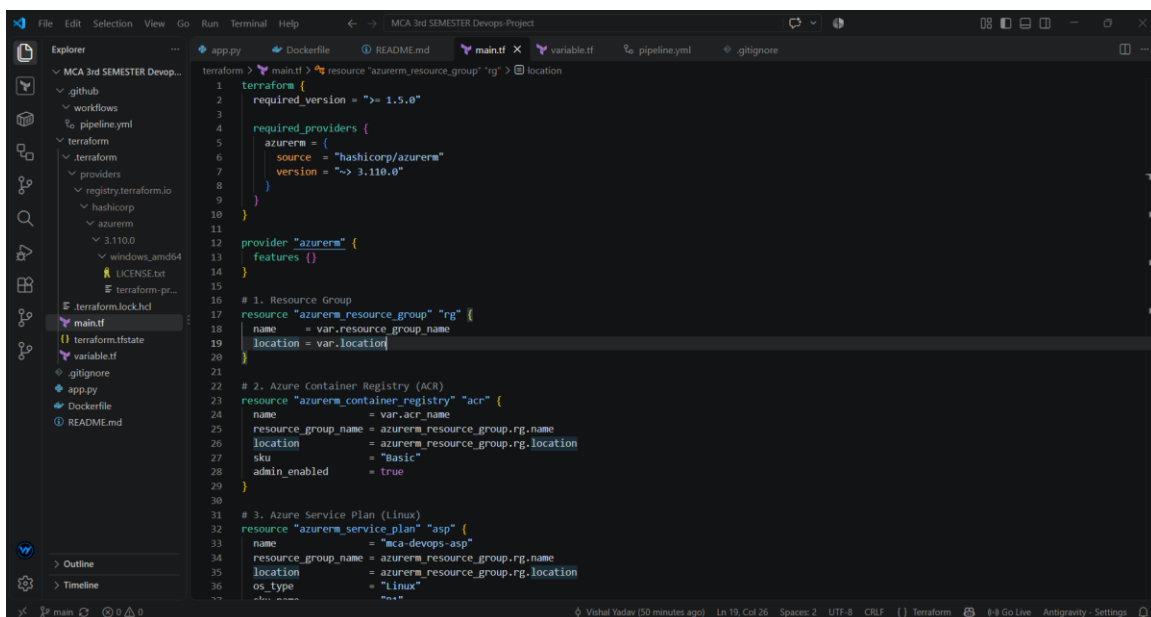


Figure 1: Project Structure in Visual Studio Code

5.6 Azure Configuration and Hosting

The supplied Azure Portal screenshot shows the Web App named mca-devops-webapp-2026 in Central India, running on Linux with container publishing. It also shows the App Service Plan and the container image reference based on mcadevopsreg2026.azurecr.io/mca-devops-app:latest. The screenshot also displays the default domain mca-devops-webapp-2026.azurewebsites.net.

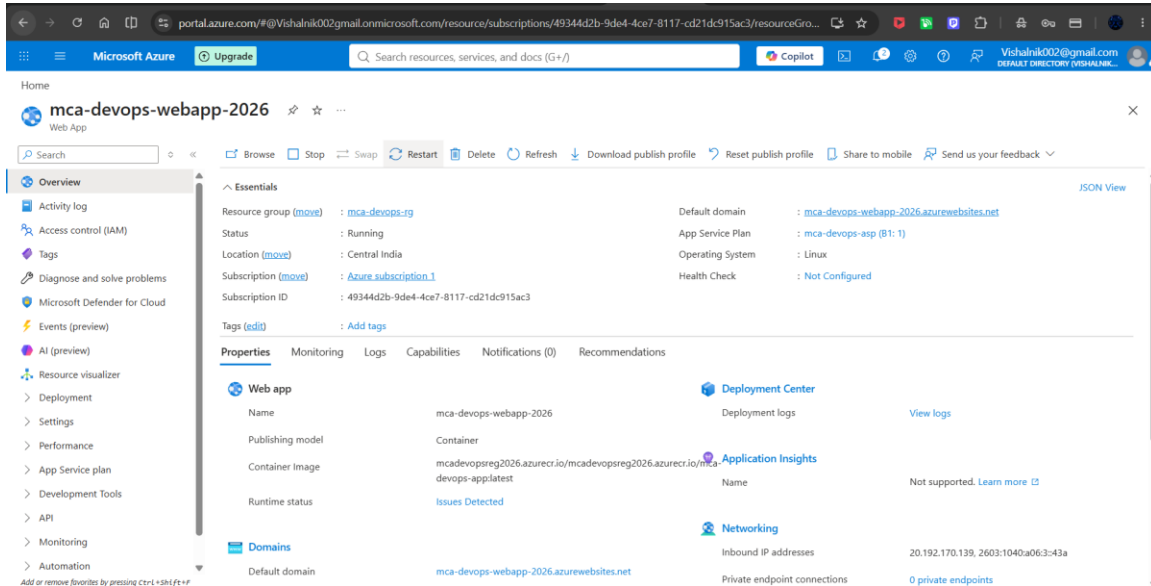


Figure 2: Azure Linux Web App Configuration and Running Resource

6. SOURCE CODE AND CONFIGURATION

The following code listings reproduce the supplied project files. The listings are included without replacing them with generic examples. The filenames and code correspond to the uploaded project materials.

6.1 app.py

Main Flask application and root route.

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return """
    <div style='text-align:center; padding-top:50px; font-family:Arial;'>
        <h1 style='color:#0078D4;'>🚀 DevOps Pipeline Optimization System Live!</h1>
        <p>Managed by: Vishal, Satyam & Rohit</p>
        <p>Status: <b>Application Successfully Deployed via Azure & Docker</b></p>
    </div>
    """

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

6.2 Dockerfile

Container image definition for the Flask application.

```
FROM python:3.9-slim
WORKDIR /app
RUN pip install flask
```

```
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

6.3 main.tf

Terraform configuration for Azure infrastructure and Linux Web App container settings.

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.110.0"
    }
  }
}

provider "azurerm" {
  features {}
}

# 1. Resource Group
resource "azurerm_resource_group" "rg" {
  name     = var.resource_group_name
  location = var.location
}

# 2. Azure Container Registry (ACR)
resource "azurerm_container_registry" "acr" {
  name                = var.acr_name
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  sku                 = "Basic"
  admin_enabled       = true
}

# 3. Azure Service Plan (Linux)
resource "azurerm_service_plan" "asp" {
  name                = "mca-devops-asp"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  os_type             = "Linux"
  sku_name            = "B1"
}

# 4. Azure Linux Web App Container Host
resource "azurerm_linux_web_app" "webapp" {
  name                = var.app_service_name
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  service_plan_id     = azurerm_service_plan.asp.id

  site_config {
    always_on = false

    application_stack {
      docker_image_name = "${azurerm_container_registry.acr.login_server}/mca-devops-app:latest"
      docker_registry_url = "https://${azurerm_container_registry.acr.login_server}"
    }
  }
}

app_settings = {
  "DOCKER_REGISTRY_SERVER_URL"      = "https://${azurerm_container_registry.acr.login_server}"
  "DOCKER_REGISTRY_SERVER_USERNAME" = azurerm_container_registry.acr.admin_username
  "DOCKER_REGISTRY_SERVER_PASSWORD" = azurerm_container_registry.acr.admin_password
  "WEBSITES_PORT"                  = "5000"
}
```

6.4 variable.tf

Terraform input variables and project-specific default resource names.

```
variable "location" {
  description = "Azure region for deployment"
  type        = string
  default     = "Central India"
}

variable "resource_group_name" {
  description = "Resource Group Name"
  type        = string
  default     = "mca-devops-rg"
}

variable "acr_name" {
  description = "Azure Container Registry Name (alphanumeric only, must be globally unique)"
  type        = string
  default     = "mcadevopsreg2026"
}

variable "app_service_name" {
  description = "Azure Linux Web App Service Name"
  type        = string
  default     = "mca-devops-webapp-2026"
}
```

6.5 pipeline.yml

GitHub Actions workflow for building and pushing the Docker image to ACR.

```
name: MCA DevOps CI/CD Pipeline

on:
  push:
    branches: [ "main" ]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.9'

      - name: Install App Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install flask

      - name: Log in to Azure Container Registry
        uses: azure/docker-login@v1
        with:
          login-server: mcadevopsreg2026.azurecr.io
          username: ${ secrets.ACR_USERNAME }
          password: ${ secrets.ACR_PASSWORD }

      - name: Build and Push Docker Image
        run: |
          docker build -t mcadevopsreg2026.azurecr.io/mca-devops-app:latest .
          docker push mcadevopsreg2026.azurecr.io/mca-devops-app:latest
```

7. CI/CD EXECUTION AND OUTPUT

7.1 GitHub Actions Successful Run

The supplied GitHub Actions screenshot shows the workflow “Fix large files and setup ACR pipeline #3” completed successfully. The build-and-deploy job displays completed steps including checkout, Python setup, dependency installation, Azure Container Registry login, Docker image build/push, and job completion. This corresponds to the steps defined in pipeline.yml.

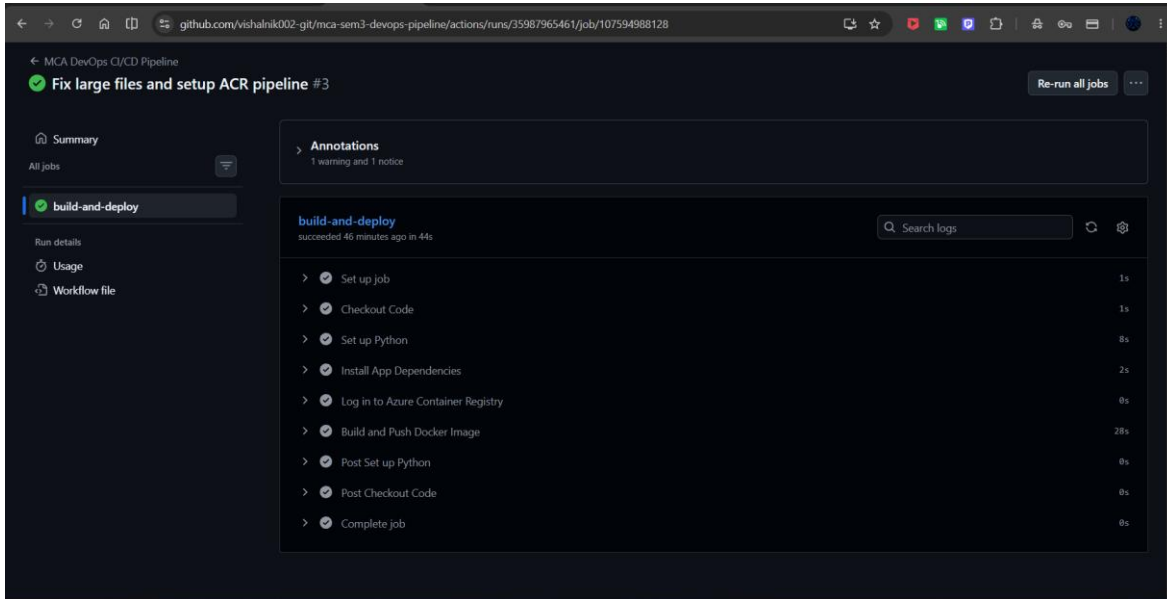


Figure 3: Successful GitHub Actions CI/CD Workflow Run

7.2 Live Web Application

The supplied browser screenshot shows the application page with the heading “DevOps Pipeline Optimization System Live!”, the team reference, and the status message indicating successful deployment via Azure and Docker. The browser address bar shows the Azure App Service domain used for the deployed application.



Figure 4: Live Flask Application on Azure App Service

8. PROJECT TESTING

8.1 Testing Objectives

- Check that the Flask application code is configured to start on port 5000.
- Check that the Dockerfile defines the required container image and startup command.
- Check that Terraform defines the required Azure resources.
- Check that GitHub Actions starts on a push to main.
- Check that the Docker image is built and pushed to ACR.
- Check that the Azure Linux Web App is configured to use the ACR image.
- Check that the deployed application is accessible through the Azure App Service domain.

8.2 Test Cases and Evidence

Test Case	Test Description	Expected Result	Evidence / Status
TC-01	Flask application configuration	Application listens on port 5000	Supported by app.py; runtime execution not separately documented.
TC-02	Docker image build	Docker image is created successfully	Observed in successful GitHub Actions run.
TC-03	Docker container configuration	Container starts the Flask application	Supported by Dockerfile CMD and exposed port; separate local-run screenshot not supplied.
TC-04	Terraform resource configuration	Azure resources are defined	Verified from main.tf and Azure resource screenshot; Terraform command output not supplied.
TC-05	GitHub Actions trigger	Workflow starts on push to main	Configured in pipeline.yml and successful run screenshot supplied.
TC-06	Build and push image	Image is pushed to ACR	Observed in successful GitHub Actions run and configured registry/image name.

Test Case	Test Description	Expected Result	Evidence / Status
TC-07	Azure Web App configuration	Web App points to ACR image	Visible in Azure Portal screenshot and defined in main.tf.
TC-08	Live application access	Application is accessible through Azure App Service	Observed in supplied live browser screenshot.

9. ADVANTAGES OF THE PROJECT

- Reduces repeated manual steps in container image delivery.
- Provides a repeatable GitHub Actions workflow for building and pushing the image.
- Uses Docker to create a consistent application runtime package.
- Uses Terraform to represent Azure infrastructure in code.
- Stores the application image in Azure Container Registry.
- Uses Azure App Service as the cloud hosting layer.
- Makes source-code changes traceable through Git and GitHub.
- Provides practical exposure to DevOps, CI/CD, containers, IaC, and cloud deployment.

10. LIMITATIONS

- The project is primarily an academic demonstration.
- The supplied GitHub Actions workflow does not include automated unit/integration testing.
- Advanced monitoring and alerting are not part of the supplied implementation.
- The current Terraform configuration defines a single primary web-app environment.
- High availability and multi-region deployment are outside the supplied scope.
- Kubernetes or another container-orchestration platform is not used.
- The pipeline shown in pipeline.yml builds and pushes the image but does not contain a direct App Service deployment action or Terraform execution step.

11. CONCLUSION

The DevOps Pipeline Optimization System demonstrates how a Python Flask application can be packaged and delivered using modern DevOps tools. The project combines Git and GitHub for source management, Docker for containerization, Terraform for Azure infrastructure configuration, GitHub Actions for automated image delivery, Azure Container Registry for image storage, and Azure App Service for hosting.

The supplied project materials and screenshots show a successful GitHub Actions run, an Azure Linux Web App configured with the project container image, and a live application page. The implementation therefore provides a practical example of connecting source control, automation, containerization, Infrastructure as Code, and cloud hosting.

The report deliberately separates the responsibilities visible in the actual files: Terraform defines the infrastructure and Web App container configuration, while pipeline.yml builds and pushes the Docker image to ACR. This keeps the technical description aligned with the submitted implementation.

12. FUTURE SCOPE

12.1 Automated Testing

Unit and integration tests can be added to the GitHub Actions workflow so that application changes are checked before an image is published.

12.2 Security Scanning

Docker images and dependencies can be scanned for known vulnerabilities before publication or deployment.

12.3 Monitoring

Azure monitoring services can be added to observe application availability, errors, and performance.

12.4 Automatic Rollback

The delivery process can be extended to return to a previously working image when a new release fails.

12.5 Multiple Environments

Separate development, testing, and production environments can be introduced.

12.6 Kubernetes

The project can be extended to use Kubernetes or Azure Kubernetes Service for container orchestration at larger scale.

12.7 Better Deployment Strategies

Deployment patterns such as blue-green or canary releases can be considered where supported by the chosen Azure architecture.

12.8 Performance Analysis

Future work can measure workflow duration, Docker build time, image push time, application startup time, and related delivery metrics.

13. REFERENCES

1. Python Documentation — <https://docs.python.org/3/>
2. Flask Documentation — <https://flask.palletsprojects.com/>
3. Git Documentation — <https://git-scm.com/doc>
4. GitHub Documentation — <https://docs.github.com/>
5. GitHub Actions Documentation — <https://docs.github.com/en/actions>
6. Docker Documentation — <https://docs.docker.com/>
7. Terraform Documentation — <https://developer.hashicorp.com/terraform/docs>
8. Microsoft Azure Documentation — <https://learn.microsoft.com/azure/>

9. Azure App Service Documentation — <https://learn.microsoft.com/azure/app-service/>

10. Azure Container Registry Documentation — <https://learn.microsoft.com/azure/container-registry/>

14. PROJECT TEAM

S. No.	Name	Admission No.	Enrollment No.	Signature
1	Vishal Yadav	25SCSE2030186	25132030877	_____
2	Satyam Tiwari	25SCSE2030189	25132030794	_____
3	Rohit Baghel	25SCSE2030104	25132030769	_____

Project Guide: Dr. Deepak Kumar Panda

Course: Master of Computer Applications (MCA), Third Semester

Academic Session: 2026–2027

University: School of Computer Application and Technology, Galgotias University, Greater Noida, Uttar Pradesh, India